User Mode Linux Redesign

Architecture, Implementation State, and Validation Evidence

Reporting cutoff: branch `umlctl-deploy`, commit `485eff5c96be`

May 14, 2026

Abstract

This report summarizes the User Mode Linux redesign as of May 14, 2026. It is both a project introduction and a state report: the first pages state the goal, the architecture, and the current validation position; later sections provide design detail, benchmark data, validation evidence, and remaining work. The central claim is that the redesign has moved beyond paper architecture: backend abstraction, static-key gates, profile work, the Rust launcher, and a substantial `kvm-v2` backend are present in-tree. The remaining formal gate is Phase J validation and upstream packaging.

1 Executive Summary

1.1 Goal and Current Position

The redesign aims to make User Mode Linux a first-class kernel research, fuzzing, and deterministic execution target again. The target is not another virtual machine monitor, not a container runtime, and not a Linux reimplementation. The target is the upstream kernel, built as UML, with swappable execution backends, runtime instrumentation gates, and profile-driven builds from one source tree.

Vision. One kernel tree produces fast production UML, sanitizer-heavy research UML, fuzzing UML, sandbox UML, library-mode UML, and time-travel UML.

Architecture. Layer 1 abstracts the trap mechanism with `struct um_backend_ops`; Layer 2 places static-key gates on hot paths; Layer 3 composes compile-time sanitizers and profile features.

Current state at May 14, 2026. Backend abstraction, static-key infrastructure, profile work, `umlctl`, selftests, and `kvm-v2` are in-tree. `kvm-v2` has cleared Phases A–H, G.1/G.2, and I; Phase J validation is in progress.

Evidence. `kvm-v2` matches seccomp on the substrate gate, reaches cpython-parity 21/21 under UP and SMP, passes 200/200 `kvm-v2` iterations in the first post-T57 two-hour soak, and is faster than seccomp on all eight benchmark hosts.

Figure 1: The report in four claims. Each claim maps back to a tracked source document or measured artefact.

1.2 Original UML Promise and Redesign Questions

Original UML made Linux itself run as a userspace process: debuggable, rootless, and able to host many isolated Linux instances without emulating a machine [1, 2, 3]. The redesign keeps that promise but answers the two criticisms that made UML feel stale for modern kernel work: userspace trap cost and incomplete multiprocessor realism.

1.3 Cutoff Snapshot

21/21	200/200	193–908x	7 series
cpython parity UP and SMP	`kvm-v2` Phase J two-hour soak	micro speedup over seccomp	staged upstream submission queue

Figure 2: Dashboard metrics at the reporting cutoff. The numbers are intentionally sparse: each is backed by a source table later in the report.

Table 1: External UML premise mapped to the redesign response.

Original premise	Why it still matters	Redesign response
Kernel as a normal host process	Rootless kernel development, CI, and reproducible debugging remain valuable where nested virtualization is unavailable.	Keep stock-host execution through ptrace/seccomp, but add KVM as a fast backend when available.
Isolated Linux instances	Sandboxes, network labs, and testbeds need cheap Linux environments without new hardware or privileged host setup.	Make sandbox a profile, not the whole architecture; choose minimal code, seccomp-only defaults, and explicit validation.
Faithful upstream kernel	Security research needs Linux behavior, not a userspace reimplementation with semantic drift.	Build the upstream kernel as UML; profiles change defaults and instrumentation, not kernel semantics.
Debuggable experimentation	Original UML made kernel debugging easier than bare metal; modern research also needs tracing, coverage, and replay.	Layer 2 static keys make observability a runtime decision; Layer 3 composes sanitizers by profile.
Known historic weaknesses	High trap cost and lack of realistic SMP limited UML as workloads and fuzzers became more demanding.	`kvm-v2` and Phase J directly target syscall cost, SMP state ownership, parity, and long-run reliability.

1.4 What Is Done, What Is Not

1.5 First Three-Page Takeaway

The redesign is best understood as a transition from "UML as a single slow compatibility target" to "UML as a profile matrix." The common lower layer is a backend contract; production can choose KVM, CI can choose `seccomp`, legacy hosts can choose ptrace, and each profile can decide whether it wants no hooks, runtime hooks, or heavy compile-time instrumentation.

The current fork has already proved the highest-risk implementation path: `kvm-v2` can run real guest userspace, handle SMP, preserve cross-task state, and beat `seccomp` on measured workloads. The remaining question is no longer whether the architecture is plausible; it is whether Phase J can demonstrate enough sustained reliability and breadth for upstream submission.

Table 2: The questions this report and deck must answer.

#	Question	Where answered
1	What is the revived UML value proposition, beyond nostalgia?	Vision, non-goals, original-premise table, and profile matrix.
2	Why is this not QEMU, Firecracker, gVisor, LKL, or a container runtime?	Non-goals, design boundary, and adjacent-work comparisons.
3	What single architecture lets fast production, research, fuzzing, sandbox, library, and time-travel coexist?	Three-layer architecture and profile sections.
4	What has actually landed, and what is still only planned?	Current-state tables, source map, and upstream queue.
5	Does `kvm-v2` preserve Linux-visible state across SMP, signals, FPU/XSAVE, faults, and task switches?	KVM design, state-audit method, recent-fix table, and validation ladder.
6	Does the performance win survive beyond a microbenchmark?	Micro, Python, and stress-tier benchmark sections.
7	What validation bar remains before upstreaming?	Phase J, Tier 2/3/LTP, and next-work sections.
8	What should reviewers challenge next?	Open-state list, remaining risks, and upstream-path sections.

Contents

Table 3: Reporting snapshot.

Reporting date	May 14, 2026
Cutoff commit	`485eff5c96be` on `umlctl-deploy`
Project corpus	253 files under `Documentation/virt/uml/redesign/`; 32 top-level `tools/testing/selftests/um/` test areas; `arch/um/backend/kvm-v2/` implementation present
Functional headline	`kvm-v2` has cleared core Phases A–H, SMP phases G.1/G.2, and the Phase I lift; formal Phase J validation remains open
Parity headline	cpython-parity is 21/21 under both UP and SMP; substrate gate is PASS=25/FAIL=3/XFAIL=3, matching seccomp
Performance headline	post-gadget `kvm-v2` beats seccomp on all eight lab hosts: micro 193–908x, Python 2.99–4.06x, stress 3.71–8.72x
Validation headline	first post-T57 two-hour soak: `kvm-v2` 200/200 PASS; aggregate 397/400 PASS, with the three failures isolated to iocheck/seccomp timeout behavior

Table 4: Status by major workstream.

ID	Workstream	State	Summary
A	Backend abstraction	DONE	`um_backend_ops` contract, ptrace/seccomp split, KUnit contract, and documentation landed.
B	Static-key hot paths	DONE	Hot-path gates, debugfs controls, and runtime flip demonstration landed.
C	Profiles and gaps	MOSTLY	Defconfigs and major sanitizer/tracing ports are largely landed; snapshot, syzkaller, and some resumed features remain.
D	KVM backend	IN PROGRESS	`kvm-v2` is functional and performant; Phase J, Tier 2/3, LTP, and upstream packaging are the active edge.

2 Goal and Vision

2.1 Problem Statement

Classic UML remains valuable because it is Linux itself compiled to run as a userspace program. That gives researchers a real kernel, real system call semantics, and a deployable target in environments where full hardware virtualization is not available. The problem is that the historical execution model does not provide the performance, feature composition, or operational discipline expected by modern kernel fuzzing and observability workflows.

The redesign sets a higher bar:

- production UML should approach native syscall cost when the host exposes KVM;
- research UML should ship with sanitizers, tracing, kprobes, ftrace, BPF, KGDB, and debug surfaces composed from one tree;
- fuzzing UML should provide KCOV, snapshot/forkserver startup, and a path toward a syzkaller `vm/uml` backend;
- sandbox UML should compile out debug surfaces and minimize host attack surface;
- time-travel and record/replay should become a profile, not a niche configuration footnote.

2.2 Non-Goals

The design deliberately avoids three tempting but wrong shapes. First, it does not reimplement Linux in another language. gVisor is important prior art for backend structure and KVM execution, but UML keeps Linux semantics by building Linux itself [4]. Second, it is not a microVM competitor to Firecracker [5]; the problem is kernel research and fuzzing fidelity, not cloud cold-start packaging. Third, it is not a unikernel-only library effort, although the library profile borrows from LKL where that model is useful [6].

2.3 Success Criteria

The project succeeds when a developer can select a profile, build it from the same tree, and get an artifact whose performance and instrumentation properties match its intended use:

Table 5: Target profile outcomes.

Profile	Intended outcome
`prod-fast`	KVM backend, no hooks, near-native syscall path.
`prod-with-hooks`	Fast production binary with hooks compiled but off; tracing can be enabled after a crash without rebuilding.
`research`	Debug-friendly UML with tracing, kprobes, sanitizers, and interactive control surfaces.
`fuzz` / `fuzz-deep`	KCOV, snapshots, sanitizer coverage, replay-oriented hooks, and syzkaller integration.
`sandbox`	Minimal debug surface and minimized host-side trust boundary.
`library`	LKL-style direct-call mode for harnesses that can sacrifice ring-split fidelity.
`time-travel`	Deterministic time and replay hooks as a first-class shipped configuration.

2.4 Why Now

The timing is favorable because recent UML work removed old prerequisite blockers: seccomp mode exists, SMP support exists, and modern kernel instrumentation is mature enough to compose around static keys and profile-specific Kconfig. The surrounding ecosystem also supplies useful models: gVisor for backend/platform design, crosvm for host process decomposition [7], syzkaller for fuzzing expectations [8], and upstream kernel policy for AI-assisted development disclosure [9].

3 Current State

3.1 Top-Down Source Reading

This report intentionally reads the project in the same order a new reviewer should read it: goal, plan, code, diary, current state, then evidence. The main source classes are:

Table 6: How the project corpus maps into this report.

Source class	Representative paths	Use in the report
Plan and vision	`00-vision.md`, `README.md`, `report-presentation/PLAN.md`	Defines the goal, non-goals, architecture narrative, and publishing shape.
Architecture	`01-architecture/`, `02-workstreams/`	Defines layers, workstreams, dependencies, profiles, and sequencing.
Code	`arch/um/include/shared/backend.h`, `arch/um/backend/kvm-v2/`, `tools/uml/uml-launcher/`	Confirms that the report is describing landed implementation, not only design.
Diary and state	`STATUS.md`, `state-audit/`, Phase J memos	Provides current truth, bug chronology, closure criteria, and remaining work.
Evidence	`tools/testing/selftests/um/`, benchmark and soak memos	Supplies measured performance, parity, soak, and gate data.

3.2 Original UML Versus This Branch

A reviewer should be able to separate inherited UML from the work this branch redesigned or implemented. I checked that boundary by diffing `origin/master...HEAD` across `arch/um`, `tools/uml`, `tools/testing/selftests/um`, and `Documentation/virt/uml`. That diff spans 688 UML-related files, with 154,501 insertions and 823 deletions. The number is a scope check, not an authorship claim: the original UML idea, the Linux kernel, and generic KVM facilities remain inherited foundations.

3.3 Landed Versus Open

The current state is not a green-field plan. The fork contains substantial code and test infrastructure:

- `arch/um/include/shared/backend.h` defines the backend contract used by host-side and kernel-side translation units.
- `arch/um/backend/seccomp/` and ptrace integration provide the non-KVM backends.
- `arch/um/backend/kvm-v2/` contains the active `kvm-v2` implementation, including vCPU, memory, syscall, exception, region, state trace, and LSTAR gadget code.
- `tools/uml/uml-launcher/` and `umlctl` provide host-side launch, supervision, gate, gate-loop, history, and deployment machinery.

Table 7: Attribution reading for the current branch.

Category	What the report means
Inherited UML and Linux	Linux-as-a-host-process, the upstream UML port, legacy ptrace execution, existing kernel subsystems, the host KVM API, and the general idea of deterministic time-travel are not claimed as new work here.
Recent upstream prerequisites	Seccomp-mode UML and upstream SMP work are treated as enabling context. The redesign builds on them; it does not present them as branch-local inventions.
Redesigned in this project	The explicit backend-ops contract, static-key hook model, profile matrix, validation ladder, staged upstream queue, and `kvm-v2` state-ownership method are the project architecture.
Implemented in this branch	`arch/um/backend/kvm-v2/`, backend contract code, profile configs and docs, snapshot/forkserver v1, the Rust launcher and `umlctl`, KVM/soak/benchmark selftests, and the report/deck workspace are branch-local artefacts.
Planned or paused	KVM-aware snapshot readiness, full record/replay, time-machine workflow, syzkaller `vm/uml`, LTP curation, and the large upstream `arch/um` series remain future work.

- `tools/testing/selftests/um/` contains benchmark, parity, gate, soak, snapshot, KVM, ftrace, kprobes, KMSAN, and process reproducer coverage.

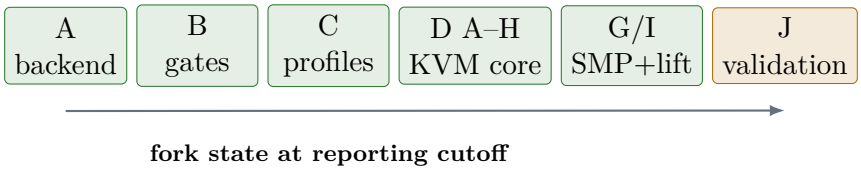

Figure 3: High-level phase state. Most architectural and implementation work has landed; Phase J is the active formal validation gate.

3.4 Current Functional Reading

3.5 Open State

The important open items are bounded:

- Phase J must still deliver a formal 24-hour continuous run, Tier 2/Tier 3 workload templates, and LTP curation.
- Snapshot and record/replay work is unblocked but paused behind Phase J.
- The T57 Phase A AVX/XSAVE fix is landed; AVX-512 exposure is deferred to a Phase B if needed.
- The LKML queue is sequenced, but the large `arch/um` series are gated on validation and upstream review strategy.

4 Architecture

4.1 Three Layers

Layer 1 isolates the mechanism that makes guest userspace run: ptrace, `seccomp`, or KVM. Layer 2 isolates optional runtime observation and policy hooks. Layer 3 isolates compiler-level instrumentation whose cost is inherent in the binary. This structure matches the host kernel's own composition

Table 8: Functional status from `STATUS.md`.

Surface	Reading	Interpretation
`init=/bin/echo`	UP + SMP yes	Basic guest entry works under `kvm-v2`.
Python smoke	UP + SMP yes	`python3 -V`, hashlib, ssl, and single imports work.
Substrate gate	PASS=25, FAIL=3, XFAIL=3	`kvm-v2` matches `seccomp` on the curated substrate gate.
cpython-parity	21/21	The old P0, Python C-extension import crashes, is functionally closed.
mt-mini T=8, ncpus=4	30/30 or better in current runs	SMP memory stress reaches the current acceptance bar.
threaded-fork-malloc	0/24000 child failures	Cross-task XMM/FPU guarantee survived the T55/T57 fixes.
wide cpython-parity	134 parity, 0 regression	Larger curated module sweep found no `kvm-v2` regression.

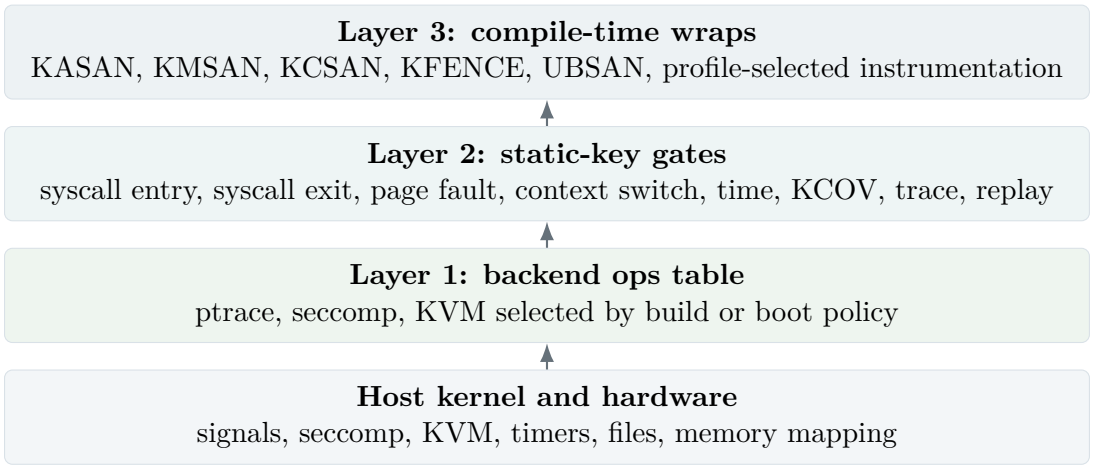

Figure 4: The architectural decomposition. Lower layers do not know about higher-layer policy; profiles choose points in the product space.

strategy: explicit lower interfaces, static keys for runtime feature toggles, and Kconfig for compile-time feature selection.

4.2 Layer 1: Backend Contract

The live contract is `struct um_backend_ops`. It has metadata, capability flags, lifecycle hooks, vCPU execution, memory region callbacks, scheduling hooks, TLB kick support, time hooks, and debug/introspection hooks. The important design choices are:

- every in-tree backend populates the whole table; unsupported cold operations return an explicit error rather than leaving NULL fields;
- single-backend builds can compile calls down to direct calls; dynamic builds can select at boot and pay one stable indirect call;
- backend capability flags replaced legacy global booleans, so host-side code asks the selected backend what stub, reaper, and syscall behavior it needs;

- per-mm memory state is owned by the backend, which lets `seccomp` keep its stub model while `kvm-v2` manages memslots and guest mappings.

Table 9: Backend contract categories.

Category	Representative ops	Why it belongs in Layer 1
Lifecycle/trap	`probe`, `init`, `vcpu_run`	Backend availability and guest execution are mechanism-specific.
Memory	`mm_create`, region add, remove, protect	ptrace/seccomp stubs and KVM memslots have different state models.
Scheduling	thread create, context switch, IPI	Guest scheduling touches backend-owned vCPU or stub state.
Time	clock read, timer set	Determinism and backend timing sources must be abstracted.
Debug	register read/write	KGDB and inspection need a common contract over different mechanisms.

4.3 Layer 2: Static-Key Hot Paths

Layer 2 is the answer to a central tension: a research build wants hooks everywhere; a production build wants the hot path cold and small. Static keys make the disabled case a patched NOP sequence and the enabled case an explicit branch to slow-path code. The off-state cost model is therefore stable and auditable.

The report treats Layer 2 as a policy plane rather than a pile of tracepoints. The same hot path can support tracing, KCOV, record/replay, time-travel, and performance event emission, provided each hook has a clear default state and each profile declares which hooks are compiled and enabled.

4.4 Layer 3: Compile-Time Instrumentation

Layer 3 includes KASAN, KMSAN, KCSAN, KFENCE, UBSAN, and related compiler- or allocator-level instrumentation. These features cannot be made free by a runtime branch. They change code generation, memory layout, or allocator behavior. The architecture therefore treats them as profile-level build choices that compose with any backend below.

4.5 A Syscall Through the Stack

Figure 5: `kvm-v2` fast syscall shape after gadget revival. Trivial syscalls avoid a host vmexit on the hot path.

5 KVM v2 Design

5.1 Model

`kvm-v2` follows the gVisor KVM-platform idea, adapted to UML: guest userspace runs at ring 3 inside a KVM guest, the UML kernel runs at ring 0 inside the same KVM guest, and ordinary syscalls enter the UML kernel through MSR_LSTAR instead of trapping to a host monitor [4, 10]. Host exits are still used for operations that truly require host participation, but the target is to keep the common syscall path in guest context.

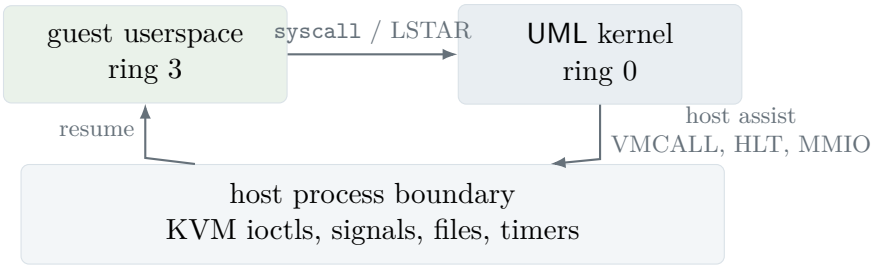

Figure 6: `kvm-v2` execution model. It is not a QEMU-style machine; it is a trap mechanism for UML itself.

5.2 Implementation Themes

The current `kvm-v2` design is organized around five hard problems:

1. **Guest entry and return.** The backend must enter guest ring 3, handle SYSCALL/SYSRET transitions, and recover when host signals interrupt critical regions.
2. **Per-vCPU and per-task state.** General registers, CR2, segment bases, FPU/XSAVE, pending events, IST frames, and KVM run state cannot leak across tasks or host CPUs.
3. **Memory and TLB state.** UML page-table changes must be reflected into KVM-visible mappings and cross-vCPU TLB state without stale translations.
4. **SMP correctness.** Host scheduling, guest scheduling, signal delivery, and KVM vCPU ownership must compose without assuming that `preempt_disable()` pins a host thread.
5. **Performance.** The backend must preserve the win from in-guest dispatch while avoiding correctness regressions from lazy state handling.

5.3 Design Detail: Main Control Surfaces

5.4 Design Detail: Fast Path Versus Slow Path

The backend now has two different syscall cost classes. Trivial syscalls covered by the LSTAR gadget stay inside the KVM guest and return directly to guest userspace. Syscalls outside the gadget set, or paths requiring host assistance, still exit to the host and run the full backend machinery. The project should keep these paths described separately:

- **Fast path:** getpid-family and clock-like calls that can be answered from guest-visible state with preserved ABI registers and no host-side work.
- **Slow path:** mmap, munmap, brk, I/O, scheduling, signal, and fault paths that need backend state or host services.
- **Recovery path:** interrupted LSTAR, page-fault stub, #NM, and signal delivery edges where the backend must rebuild user-visible architectural state exactly.

Table 10: `kvm-v2` implementation surfaces to understand before reviewing patches.

Surface	Representative code	Design role
Initialization and capabilities	`init.c`, `ops.c`, `Kconfig`	Probe host KVM support, wire backend ops, expose configuration choices.
vCPU execution	`vcpu.c`	Owns KVM_RUN, register marshaling, migration pinning, FPU capture, dispatch, and per-vCPU state.
Syscall path	`syscall_trap.c`, `lstar_gadget.S`	Handles LSTAR entry, in-guest fast syscalls, slow-path exits, and return-state repair.
Memory and regions	`memslot.c`, `region.c`, backend memory ops	Maps UML memory changes into KVM-visible regions and memslot policy.
Exceptions	`exception.c`	Installs and handles IDT/TSS/IST paths, page faults, and special recovery paths.
State tracing	`state_trace.[ch]`	Gives low-overhead, static-key-controlled visibility into state ownership bugs.
Tests	`test_marshal.c`, `test_byteshape.c`	Protect register ABI, gadget byte shape, and internal assumptions.

5.5 State-Audit Method

The state-audit directory is part of the design, not just project diary. It converted a vague SMP corruption class into a finite audit: 149 state items, 50+ operations, an ownership matrix, suspect register audits, runtime tracing, and ranked findings.

Table 11: State-audit layers.

Layer	Artefact	Contribution
1	State inventory	Enumerates register, control, segment, MSR, FPU, event, descriptor, per-task, per-vCPU, and per-mm state.
2	Operations inventory	Lists hardware, KVM, backend, scheduler, and signal operations that read or write state.
3	Ownership matrix	States who owns each state item at each transition.
4	Suspect register audits	Tracks RBP, RAX, RDX, FS_BASE and similar corruption signatures.
5	Toolkit	cscope, ripgrep, ast-grep, bpftrace, ftrace, and parser recipes.
6–8	Findings and fixes	Converts candidates into patches and regression matrices.

The most important fix from this arc was the SMP-T13 `migrate_disable()` finding. UML was using `preempt_disable()` to pin access to per-host-CPU vCPU state, but the build did not have preemption count semantics, so the call did not actually prevent migration. Replacing that assumption with `migrate_disable()` removed a structural cross-vCPU ownership violation.

5.6 Recent Fixes

5.7 Design Boundary

The current backend should be presented upstream as a backend for UML, not as a general VMM. The VMM-like machinery exists only to make UML execution fast and faithful. That framing matters: it keeps device emulation, boot firmware, hardware discovery, and cloud microVM expectations out of scope.

Table 12: Selected recent `kvm-v2` correctness and performance closures.

Item	Closure	Why it matters
SMP-T41	Recover user RAX in EINTR-mid-PF-stub	Closed the true mt-mini byte-zero residual.
SMP-T54	SOCK_CLOEXEC and `umlctl` sysrq halt	Removed worker fd leak and init shutdown hang class.
SMP-T55	Per-vCPU FPU-dirty epoch	Restored perf-py-startup gate while preserving cross-task FPU guarantees.
SMP-T57 Phase A	CR4.OSXSAVE, XCR0=0x7, CPUID AVX/XSAVE unmask	Turned stress-ng AVX #UD from a fragile mask problem into supported guest architectural state.
Gadget revival	LSTAR in-guest fast path	Collapsed trivial syscall cost from host-exit scale to roughly 90 cycles on measured hosts.

6 Profiles and Validation

6.1 Profile Matrix

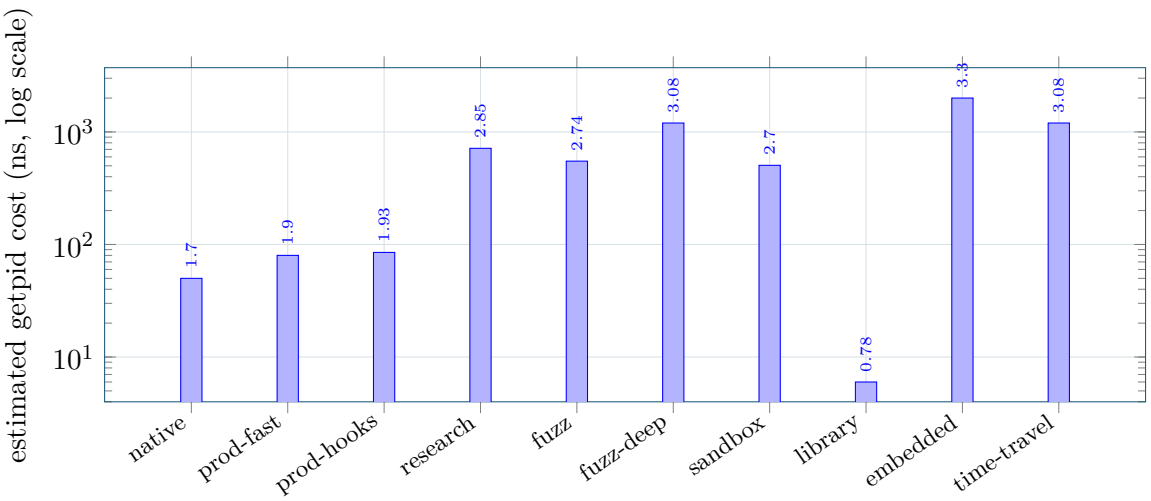

Figure 7: Profile cost model from the redesign docs, anchored to a 50 ns normal Linux `getpid` baseline. The value of profiles is the ability to span direct-call library mode through heavily instrumented research and time-travel builds without forking the tree.

6.2 Validation Ladder

6.3 Phase J Soak

The three failures were isolated to iocheck/seccomp timeout behavior under two-worker contention. They were not data miscompares and not a `kvm-v2` regression. The meaningful reading is that daemon mode, threshold-trip behavior, Wilson summary generation, and sustained `kvm-v2` workload rotation all worked.

Table 13: Shipped profile intent.

Profile	Backend default	Hooks	Sanitizers	Purpose
`prod-fast`	KVM > seccomp	none	none	Minimum overhead production target.
`prod-with-hooks`	KVM > seccomp	compiled, off	none	Production artifact with post-failure observability.
`research`	seccomp	trace, kprobes	KASAN, UBSAN	Debug and exploit-reproduction profile.
`fuzz`	seccomp	KCOV, snapshot	KASAN	syzkaller and harness fuzzing.
`fuzz-deep`	seccomp	KCOV, snapshot, replay	KASAN, KCSAN	deeper race/replay investigations.
`sandbox`	seccomp-only	none compiled	none	Minimized TCB and debug surface.
`library`	direct call	n/a	optional KASAN	LKL-style harness integration.
`embedded`	ptrace	none	none	Old-host fallback.
`time-travel`	seccomp	trace, time-travel	KASAN	deterministic clock and replay.

7 Performance and Benchmarks

7.1 Benchmark Method

The post-gadget cross-host benchmark used one kernel build, one benchmark bundle, the same orchestrator, performance governor, and the same three tiers across eight lab hosts:

- **micro**: `getpid-loop`, RDTSC-bracketed, pure trap-mechanism cost.
- **py**: canned Python startup/mixed-syscall workload.
- **stress**: mt-mini SMP T=8, ncpus=4, strict memory verification.

7.2 Native Baseline Context

The cross-host charts below use same-host speedup over `seccomp` because that ratio cancels most CPU-frequency and microarchitecture noise. For intuition, the profile cost model also carries a normal Linux `getpid` reference of 50 ns, shown here as 1.0x.

7.3 Micro Tier

The micro result is the clearest evidence that the LSTAR gadget changed the backend's cost class. Before the gadget, a trivial syscall paid a host-exit round trip. After gadget revival, the measured `kvm-v2` cost is roughly 87–153 cycles on the tested machines, while seccomp remains tens of thousands of cycles per call.

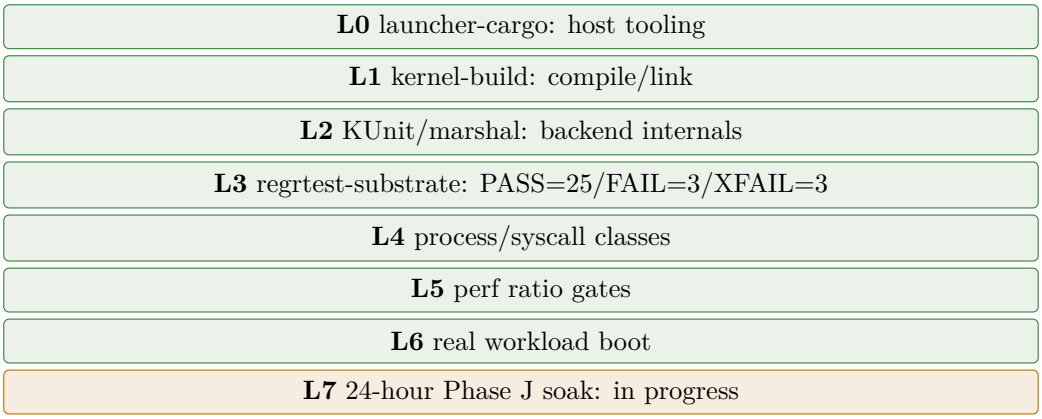

Figure 8: Validation ladder. Short gates protect the inner loop; L7 is reserved for milestone candidates.

Table 14: First post-T57 Phase A sustained soak, 2026-05-14.

Workload	Backend	n	pass	fail	pass rate
memcheck	`kvm-v2`	60	60	0	100.00%
memcheck	seccomp	60	60	0	100.00%
iocheck	`kvm-v2`	60	60	0	100.00%
iocheck	seccomp	60	57	3	95.00%
stress-ng	`kvm-v2`	40	40	0	100.00%
stress-ng	seccomp	40	40	0	100.00%
tier1-pylibs	`kvm-v2`	40	40	0	100.00%
tier1-pylibs	seccomp	40	40	0	100.00%
`kvm-v2` **aggregate**		200	200	0	**100.00%**
Overall aggregate		400	397	3	**99.25%**

7.4 Application and Stress Tiers

The Python tier matters because it turns the micro win into an application-level win: 2.99–4.06x faster than seccomp. The stress tier matters because it combines performance with correctness: the run reported zero strict memory failures and zero verify failures on every host.

7.5 Detailed Cross-Host Table

7.6 Performance Interpretation

The performance story has three layers:

- **Mechanism win.** The micro tier proves that in-guest fast-path dispatch removes most seccomp signal/stub cost for trivial syscalls.
- **Workload win.** The Python tier proves that real userland startup benefits from those trivial syscalls becoming cheap.
- **Concurrency win.** The stress tier is mmap/page-fault heavy, so the gadget is less directly relevant; the continuing 3.71–8.72x speedup comes from the broader vCPU and SMP design.

The correct benchmark unit is the ratio against seccomp on the same host, not the raw absolute latency across machines. That choice removes most CPU-frequency and microarchitecture noise and keeps the claim tied to a backend decision.

Table 15: `getpid` context relative to a normal host syscall baseline.

Path	Model cost	Relative to normal	Reading
Normal Linux host syscall	50 ns	1.0x	Human-scale baseline for a trivial syscall on the host.
`kvm-v2 prod-fast`	80 ns	1.6x	Same cost class as the host baseline for gadget-covered calls.
`kvm-v2` cross-host gadget	87–153 cycles	mechanism tier	Raw cycles vary by host; the robust claim is same-host speedup over `seccomp`.
`seccomp` profiles	505–1,200 ns	10–24x	Expected for research, fuzz, sandbox, and time-travel profiles where instrumentation or isolation dominates.
ptrace fallback	2,000 ns	40x	Old-host fallback, useful for compatibility rather than speed.
direct library call	6 ns	0.12x	LKL-style mode removes the syscall boundary; it is not a faithful ring-split kernel target.

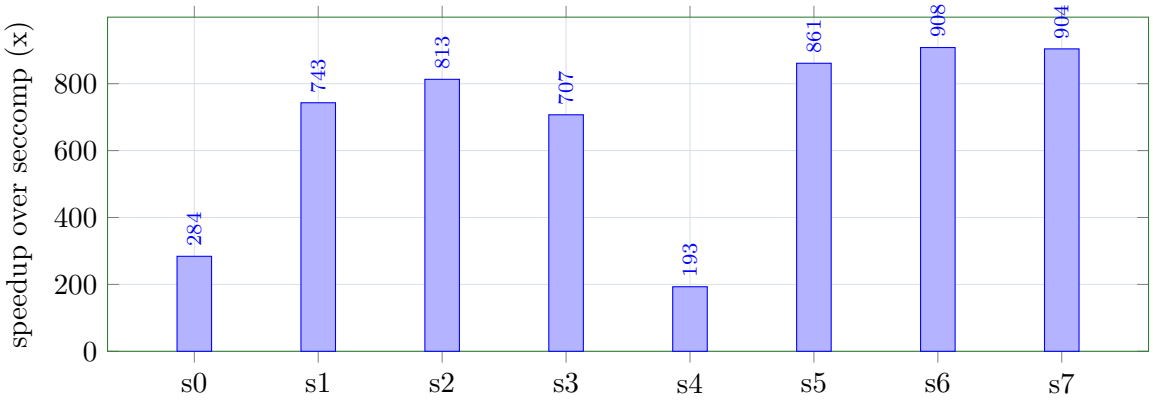

Figure 9: Post-gadget micro speedup. Trivial syscall dispatch is 193–908x faster than seccomp across the eight hosts.

8 Remaining Work and Upstream Path

8.1 Immediate Work

8.2 Snapshot, Replay, and Time-Travel Status

The “time machine” story should be presented as a staged roadmap, not as a finished product. The branch has real snapshot/forkserver plumbing and an explicit time-travel profile, but full record/replay is still future work.

8.3 Residual Risks

Two narrow residual flake classes are tracked as P2: an mt-mini page-recycling class and a high-yield mt-yieldonly RIP-corruption class. They are not current Phase J blockers and do not block snapshot/record-replay resumption, but they should remain visible in the report because they represent exactly the kind of concurrency state drift the state-audit method was built to catch.

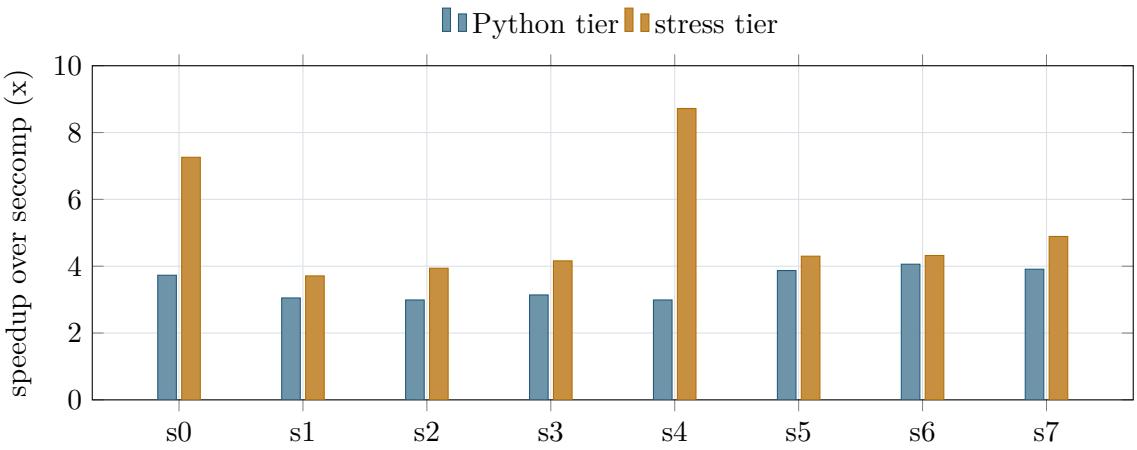

Figure 10: Application and stress tiers. `kvm-v2` wins on every host in both tiers.

Table 16: Post-gadget speedup over seccomp.

Host	CPU	micro	Python	stress
s0	i9-12900K	284x	3.73x	7.26x
s1	Xeon E3-1225 v6	743x	3.05x	3.71x
s2	Xeon E3-1225 v5	813x	2.99x	3.94x
s3	Xeon W-2123	707x	3.14x	4.16x
s4	i5-12600K	193x	2.99x	8.72x
s5	Ryzen 7 7840HS	861x	3.87x	4.30x
s6	Ryzen 7 7840HS	908x	4.06x	4.32x
s7	Ryzen 7 7840HS	904x	3.91x	4.89x

8.4 Upstream Queue

The upstream strategy is deliberately staged. Small, independent subsystem patches go first. The backend abstraction is the tentpole. Static-key hot paths and profile work follow. The KVM backend is last because it is the largest technical and review burden.

8.5 Main Message for Reviewers

The report should not ask reviewers to accept the whole future at once. It should make three narrower claims:

1. UML already has multiple backend mechanisms; making the backend contract explicit is a cleanup and an enabling move.

2. static-key hooks are the right kernel-native way to make instrumentation runtime-flippable without charging every profile.

3. the `kvm-v2` evidence is strong enough to justify continued validation and an eventual upstream RFC, but the large KVM series should wait until Phase J is formalized.

A Source Map

Table 17: Next work at the reporting cutoff.

Item	State	Notes
Phase J 24h continuous	IN PROGRESS	Daemon exists and passed a two-hour short run; formal 24-hour run remains.
Tier 2 / Tier 3	IN PROGRESS	Designs exist; templates and bootstrap flow still need implementation.
LTP curation	PLANNED	Needs KEEP/SKIP list and a UML-appropriate runner.
KVM-aware snapshot readiness	PLANNED	Unblocked after Python parity closure; resumes after Phase J.
Record/replay/time machine	PLANNED	Time-travel profile exists; full replay workflow remains paused.
AVX-512 Phase B	PLANNED	Optional follow-up for the remaining T57 vm-method failures.

Table 20: Primary source material used for this report.

Path	Role in the report
`00-vision.md`	Vision, non-goals, success criteria, stakeholder framing.
`README.md`	Top-level project layout and high-level architecture summary.
`STATUS.md`	Current state, phase ledger, workstream status, open items.
`01-architecture/three-layers.md`	Backend/gate/wrap architecture.
`01-architecture/data-flow.md`	Per-profile syscall cost model and profile comparison.
`02-workstreams/README.md`	Workstream decomposition and critical path.
`02-workstreams/D-kvm-backend/README.md`	KVM backend motivation and design framing.
`02-workstreams/D-kvm-backend/state-audit/`	State ownership audit, bug chronology, and recent fixes.
`02-workstreams/D-kvm-backend/bench-cross-host-2026-05-04-postgadget.md`	Cross-host benchmark data and methodology.
`02-workstreams/D-kvm-backend/phase-J-soak-2h-2026-05-14.md`	Latest soak result and interpretation.
`03-profiles/README.md`	Profile matrix and estimated per-profile cost model.
`05-validation/`	Validation tiers, benchmarks, conformance, CI, and quality strategy.
`upstream-patches/SUBMISSION-QUEUE.md`	Sequenced LKML/upstream queue.
`tools/testing/selftests/um/`	Concrete gate, parity, soak, benchmark, and reproducer harnesses.
`tools/uml/uml-launcher/`	Host-side launcher and `umlctl` machinery.

Table 18: Snapshot, replay, and time-travel state.

Surface	State	Reading
Snapshot/forkserver v1	**DONE**	C-09 landed a cooperative `um_snapshot_ready()` point, AFL-compatible fds 198/199, `/sys/kernel/um/state_version`, launcher fd plumbing, and `snapshot-smoke` coverage.
v1 ceiling	known limit	Status is reported as zero, payloads must be short and non-blocking, there is no on-disk snapshot, and v1 is not SMP-capable. These are documented limits, not hidden claims.
KVM-aware readiness	**PLANNED**	Issue #168 is unblocked by Python parity closure but intentionally sits behind Phase J.
Time-travel profile	profile exists	`uml/time-travel` selects seccomp-only, UP, `CONFIG_UML_TIME_TRAVEL_SUPPORT`, and debug surfaces. The runtime hook gate exists; today its slow path is still a counter stub.
Record/replay/time machine	**PLANNED**	Issue #169 covers the first-class deterministic replay extension. The event log, replay consumer, and user workflow are not implemented yet.
Syzkaller binding	planned external	The in-tree forkserver surface is pinned; the remaining `pkg/vm/uml` Go driver belongs in the syzkaller repository.

Table 19: Sequenced upstream queue.

#	Series	Size	Subsystem	Status
1	`bpf-hygiene-v1`	2 patches	BPF	ready
2	`kmsan-arch-callback-rfc`	1 RFC patch	mm/kmsan	ready
3	`ftrace-notrace-generic-v1`	1 patch	tracing/ftrace	prepared
4	backend ops abstraction RFC	about 12 patches	arch/um	to write
5	static-key hot paths	about 6 patches	arch/um	to write
6	per-profile C-series	about 20 patches	arch/um plus subsystems	blocked by Series 7 ordering
7	KVM backend series	about 15 patches	arch/um plus KVM	blocked by Phase J

B Reporting Data Files

The report and deck share normalized data under `Documentation/virt/uml/redesign/report-presentation/data/`. At this cutoff the important files are:

- `bench_speedups.csv` and `bench_ratios.csv` for the eight-host post-gadget benchmark.
- `soak_summary.csv` for the 2026-05-14 two-hour Phase J soak.
- `profile_getpid.csv` for profile cost visualization.
- `workstreams.csv`, `validation_ladder.csv`, `upstream_queue.csv`, and `status_kpis.csv` for status tables.

References

[1] Jeff Dike. “A User-mode Port of the Linux Kernel”. In: *Proceedings of the 4th Annual Linux Showcase and Conference*. USENIX / ALS 2000. 2000.

[2] *User Mode Linux HOWTO*. https://docs.kernel.org/virt/uml/user_mode_linux_howto_v2.html. Linux kernel documentation.

[3] *User Mode Linux*. https://user-mode-linux.sourceforge.net/. Project homepage.

[4] *gVisor Architecture Guide: Platforms*. https://gvisor.dev/docs/architecture_guide/platforms/. gVisor documentation.

[5] *Firecracker MicroVM*. https://firecracker-microvm.github.io/. Project documentation.

[6] *Linux Kernel Library*. https://github.com/lkl/linux. Project repository.

[7] *crosvm Book: Architecture Overview*. https://crosvm.dev/book/architecture/overview.html. Project documentation.

[8] *syzkaller*. https://github.com/google/syzkaller. Project repository.

[9] *Linux Kernel Documentation: Coding Assistants*. https://docs.kernel.org/process/coding-assistants.html. Linux kernel documentation.

[10] *KVM API Documentation*. https://docs.kernel.org/virt/kvm/api.html. Linux kernel documentation.